

Caracal - plano executavel

Modelo especialista em ciberseguranca, 3 bilhoes de parametros. 5 fundadores, pool aberto, todos treinam tudo. Ship v1 ate 21 jul 2026 (4 sprints). Atualizado 22 jun. Documento pra imprimir e levar pra reuniao com o time.

INDICE

- 1. Alvo e numeros
- 2. Os 5 modulos de Caracal (sem dono fixo)
- 3. Arquitetura (diagrama)
- 4. Datasets concretos (28 fontes)
- 5. Trilha de aprendizado (docs / github / videos)
- 6. Ferramentas e contas a abrir
- 7. Sprints sem-a-sem
- 8. 35 tarefas (com prioridade)
- 9. Compute e creditos
- 10. Riscos e mitigacoes
- 11. Harness recursivo (6 loops self-improving)
- 12. Estrategia Kaggle: 5 contas relay race
- 13. Como usar (instrucoes por fundador)
- 14. Mensagem pronta pra enviar aos fundadores
- 15. Glossario

1. Alvo e numeros

COISA	VALOR
Modelo final	Caracal-1 v1, 3B parametros, base Qwen2.5-Coder-3B-Instruct (Apache 2.0)
Data de envio	21 jul 2026 (4 sprints de 7 dias)

Alvo CyberGym 1507	40-50% pass@5 (frontier GPT-5 fez 11%)
Alvo zero-days	10-15 em 30 OSS repos held-out 2026
Alvo custo	\$700 por zero-day amortizado, \$30-35k total v1
Distribuicao	HuggingFace Hub <code>iterate-labs/caracal-1-v1</code>
Paper preprint	arXiv ate 14 ago 2026
Org GitHub	github.com/iterate-labs-ai

Pool aberto, todos treinam tudo. Sem dono fixo de modulo. Quem termina uma tarefa do pool comenta `claim` na proxima. Cada um aprende todos os modulos ao longo dos sprints. Rotacao real = redundancia + ownership coletivo.

2. Os 5 modulos de Caracal (sem dono fixo)

Caracal-1 v1 e composto por 5 modulos LoRA especialistas sobre um Caracal Base compartilhado. Cada modulo e LoRA $r=32$ de 130MB. Conversam via protocolo JSON estruturado. **Qualquer fundador treina qualquer modulo** conforme tarefa disponivel no pool. Ao longo de 4 sprints todos terao tocado em todos os modulos.

Modulo 1 · Recon

130MB LoRA $r=32$ · embodied via gdb live · STRETCH inicial, MUST sprint 2

Scout codebase, prioriza alvos. Le call graphs estaticos. **Novidade radical:** opera tambem via comandos gdb/mi reais em sessao live. Le state runtime, valores de variaveis, backtrace. Aprende debugging como skill core, nao chuteira chain-of-thought.

Modulo 2 · Hypothesizer

130MB LoRA $r=32$

Enumera 3 hipoteses de bug class por target. Output estruturado: tipo CWE, primitiva de exploit, dificuldade estimada, primitivas necessarias do banco. Treinado em writeups de CTF + Project Zero + Trail of Bits.

Modulo 3 · Crafter

130MB LoRA $r=32$ · state machine policy

Escreve PoC concreto, segue state machine rigida: ANALYZE → INSTRUMENT → EXECUTE → REFINE. Recompensa parcial por transicoes validas. Counterfactual mutator (gera "mutacao minimal que NAO triggaria") amarra reward causalmente.

Modulo 4 · Validator

130MB LoRA r=32 · SMT bounded Z3

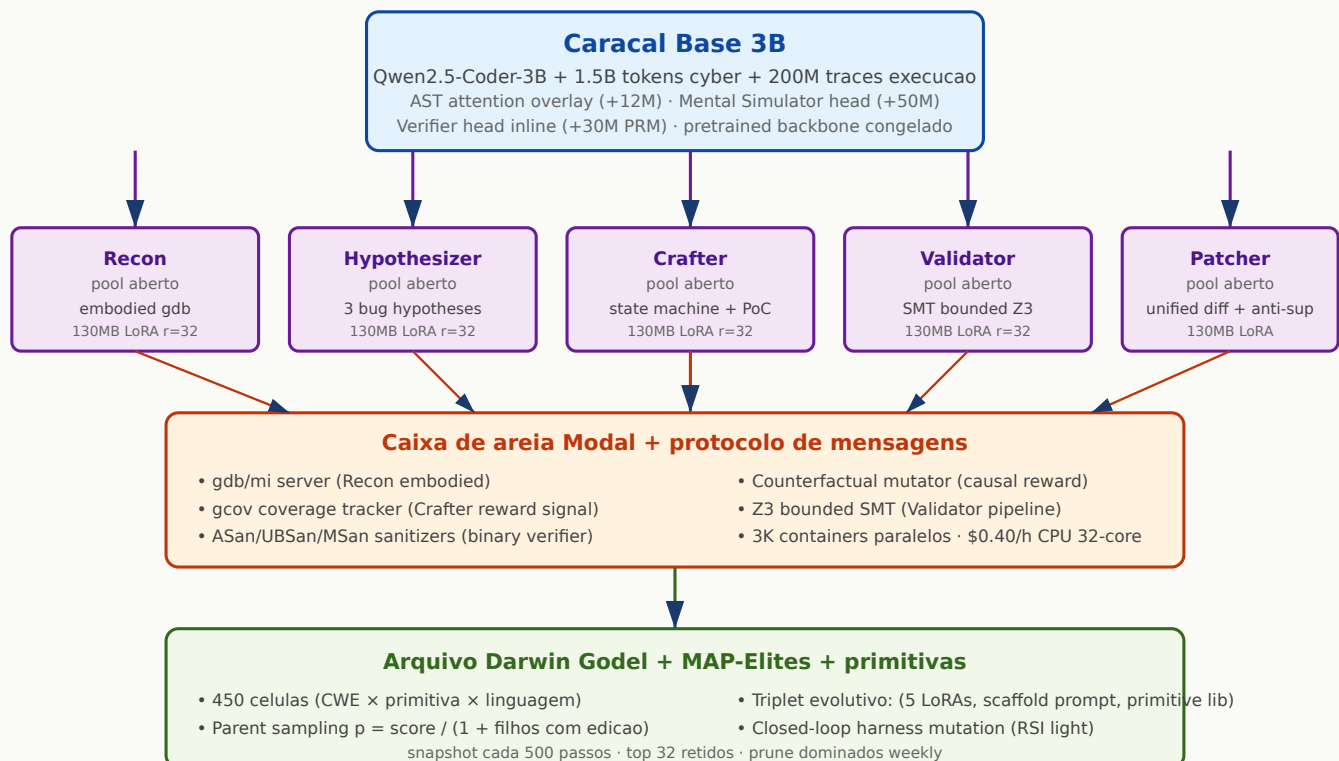
Gera specification formal do bug em Hoare-lite. Invoca Z3 bounded (depth 5) pra verificar match entre spec e PoC. Causal grounding por **prova**, nao so empirico. Reward por consistencia spec ↔ PoC ↔ execucao.

Modulo 5 · Patcher

130MB LoRA r=32 · anti-supressao check

Gera unified diff. Anti-supressao check (re-roda sanitizer em 10 inputs adjacentes pra pegar catch-and-ignore). Preserva coverage existente. Reward por patch valido + nao quebrar testes + nao suprimir bugs adjacentes.

3. Arquitetura (diagrama)



5 LoRAs especialistas compartilham um Caracal Base. Sandbox roda comandos especializados (gdb, coverage, sanitizer, SMT). Archive evolutivo guarda tudo: pesos + scaffold + primitivas.

4. Datasets concretos (28 fontes)

Tudo decontaminado contra CyberGym 1507 e held-out 30 CVE 2026 via pipeline C6 (hash + 8-gram + cosseno).

4.1 Continued pretrain Caracal Base 3B (~1.7B tokens)

FONTE	TAMANHO	ACESSO	LICENCA
NVD CVE feed completo	~250M tokens	nvd.nist.gov/vuln/data-feeds · JSON 2002-2026	CC0
GitHub patch diffs CVE-tagged	~400M tokens	GitHub API + commit SHAs do NVD	por repo
OSS-Fuzz reproducers historicos	~200M tokens	google/oss-fuzz-vulns	Apache 2.0
Project Zero blog	~50M tokens	Scrape <code>googleprojectzero.blogspot.com</code>	fair use research
Trail of Bits blog	~30M tokens	Scrape <code>blog.trailofbits.com</code>	fair use research
AFL++ docs + research papers	~20M tokens	github.com/AFLplusplus + arXiv	Apache 2.0 + arXiv
PrimeVul	~150M tokens	HF <code>secmlr/PrimeVul</code>	MIT
BigVul	~200M tokens	HF <code>bstee615/bigvul</code> · Zenodo	CC-BY-4.0
DiverseVul	~150M tokens	HF <code>bstee615/diversevul</code>	MIT
Secure-Instruct synthetic 2026	~50M tokens	arXiv 2510.07189 + HF release	MIT
OSS-Fuzz execution traces (Tier 3)	~200M tokens	Gerar com gcov + ASan dumps de 100K runs	derivado de OSS-Fuzz
NIST Juliet Test Suite (NOVO)	~80M tokens	samate.nist.gov/SARD · 64K casos sinteticos rotulados	publico US Gov

SARD (Software Assurance Reference Dataset)	~100M tokens	samate.nist.gov/SARD/testsuite.php	publico US Gov
Devign	~60M tokens	HF antolin/devign · arXiv 1909.03496	MIT
VulDeePecker (CWE-119, CWE-399)	~40M tokens	CGCL-codes/VulDeePecker	MIT
CrossVul	~80M tokens	Zenodo · multi-language (C, C++, PHP, Java)	CC-BY
MegaVul	~120M tokens	HF Icyrockton/MegaVul	MIT
Reposvul	~100M tokens	arXiv 2401.13169 · 4K real-world CVEs · repo level	MIT
PatchDB	~50M tokens	github.com/SunLab-GMU/PatchDB · 12K security patches	academic
SVEN (SARD-derived hardened)	~30M tokens	arXiv 2302.05319 · SARD curado para fine-tune	MIT

4.2 SFT estagio 2 (~35K exemplos curados)

FONTE	QUANTIDADE	ACESSO
CTF writeups (multiplas fontes)	~8K	Scrape ctftime.org + writeups.io + GitHub topic ctf-writeups
pwn.college labs + solucoes	~2K	github.com/pwncollege
Cyber-Zero ReAct traces	~10K	HF dataset Cyber-Zero (arXiv 2508.00910) · so SFT, NUNCA RL substrate
Hand-curated PoC+patch pairs (curadoria interna)	~5K	Bug bounty disclosures + HackerOne reports publicos via API
gdb session traces (Recon embodied)	~5K	Scrape ippsec + LiveOverflow + John Hammond videos + CTF writeups
HackerOne disclosed reports (NOVO)	~3K	hackerone.com/hacktivity via API · filter publico
ExploitDB metasploit modules (NOVO)	~2K	offensive-security/exploitdb

4.3 Treino por reforco e avaliacao

CONJUNTO	TAREFAS	USO
CyberGym 1507 (Berkeley, ICLR 2026)	1507 vulns / 188 projetos	Avaliacao primaria · NUNCA em treino
CyberGym subset 200 (curado por nos)	200 vulns <50 LOC patch, <30min wall	Treino por reforco
30 CVE held-out 2026 (curado C5)	30 OSS pequenos pos-jan/2026	Zero-day hunt headline · NUNCA visto
CyBench (40 CTFs)	40 CTF profissionais	Avaliacao secundaria
InterCode-CTF	variavel	Treino interactive
HumanEval+ / MBPP+	~600 problemas Python	Regression guard (nao perder capability geral)
SWE-Bench-Verified Lite 50	50 instancias	Sanity check geral
CTF-Dojo (NOVO)	variavel	arXiv 2508.18370 · treino agent finding vulns via CTF
ZeroDayBench (NOVO)	variavel CVSS≥7.0	arXiv 2603.02297 · ICLR 2026 Workshop · discovery + remediation
CVE-Bench (NOVO)	real-world	repair eval
BountyBench (NOVO)	bug bounty real	discovery validation
CAIBench (NOVO)	meta-bench	arXiv 2510.24317 · agregador cross-benchmark

5. Trilha de aprendizado (docs / github / videos)

Todos vao treinar todos os modulos. Cada area vem com 4-6 recursos comecando do basico. Marcar leitura no docs/learned.md de cada repo.

A · Fundamentos Transformer + LLM

Comecar aqui se nunca implementou um Transformer.

RECURSO	LINK	TIPO
---------	------	------

Andrej Karpathy · "Let's build GPT from scratch"	youtube.com/watch?v=kCc8FmEb1nY	video
Karpathy · nanoGPT repo	karpathy/nanoGPT	github
Annotated Transformer · Harvard NLP	nlp.seas.harvard.edu/annotated-transformer	blog
3Blue1Brown · Neural Networks (series 1-7)	youtube playlist	video
Lilian Weng · Transformer family	lilianweng.github.io	blog
HuggingFace LLM Course	huggingface.co/learn/llm-course	curso

B · LoRA / PEFT / fine-tuning eficiente

Necessario antes de treinar qualquer modulo Caracal.

RECURSO	LINK	TIPO
LoRA paper · Hu et al 2021	arXiv 2106.09685	paper
QLoRA paper · Dettmers et al 2023	arXiv 2305.14314	paper
HuggingFace PEFT docs	huggingface.co/docs/peft	docs
Unsloth · notebooks oficiais	unslothai/unsloth	github
Sebastian Raschka · LoRA from Scratch	magazine.sebastianraschka.com	blog
TRL docs · SFTTrainer	huggingface.co/docs/trl	docs

C · Reinforcement Learning de LLM (GRPO/DAPO/PPO)

Core do treino dos modulos Caracal. Foco em GRPO + DAPO.

RECURSO	LINK	TIPO
DeepSeek-R1 paper	arXiv 2501.12948	paper
Will Brown · GRPO Demystified	willccbb.github.io	blog
verl framework docs	volcengine/verl	github
OpenAI Spinning Up RL	spinningup.openai.com	curso
HuggingFace Alignment Handbook	huggingface/alignment-handbook	github

OpenPipe ART (agent RL)	OpenPipe/ART	github
Yannic Kilcher · DeepSeek-R1 explained	youtube search "Kilcher DeepSeek-R1"	video

D · Ciberseguranca basics (binary exploitation + web)

Pra entender que tipo de bug estamos cacando. Pegada hands-on.

RECURSO	LINK	TIPO
LiveOverflow · Binary Exploitation playlist	youtube.com/c/LiveOverflow	video
pwn.college · curriculo gratis Arizona State	pwn.college	curso
ippsec · HackTheBox writeups	youtube.com/c/ippsec	video
John Hammond · CTF + malware	youtube.com/c/JohnHammond010	video
"Hacking: The Art of Exploitation" · Jon Erickson	amazon · classico	livro
"Practical Binary Analysis" · Dennis Andriesse	nostarch.com	livro
OWASP Top 10	owasp.org/Top10	docs
CWE/CAPEC database · MITRE	cwe.mitre.org	docs
Project Zero blog	googleprojectzero.blogspot.com	blog

E · gdb + debugging embodied (Recon module)

Critico pra modulo Recon. gdb skill rara mas alta alavancagem.

RECURSO	LINK	TIPO
gdb manual · GNU	sourceware.org/gdb	docs
gdb/mi (Machine Interface) docs	gdb/mi manual	docs
pwndbg (gdb plugin pra exploit dev)	pwndbg/pwndbg	github
GEF (GDB Enhanced Features)	hugsy/gef	github
pwntools (Python exploit dev)	Gallopsled/pwntools	github
LiveOverflow · gdb tutorials	youtube search "LiveOverflow gdb"	video

F · Sandbox + container security

Caixa de areia hardened pra rodar código não-confiável.

RECURSO	LINK	TIPO
gVisor docs	gvisor.dev	docs
Firejail docs	firejail.wordpress.com	docs
nsjail (Google sandbox)	google/nsjail	github
seccomp BPF tutorial	man7.org/seccomp	docs
Modal docs · serverless containers	modal.com/docs	docs
OSS-Fuzz docs · build infra	google.github.io/oss-fuzz	docs

G · SMT + Z3 (Validator module)

Verificação formal pra Validator module. Z3 bounded depth.

RECURSO	LINK	TIPO
Z3 tutorial · Microsoft Research	microsoft.github.io/z3guide	docs
Z3 Python API	z3prover.github.io	docs
"The Z3 Theorem Prover" paper	arXiv search "Z3 theorem prover"	paper
angr · symbolic execution	angr/angr	github
KLEE · symbolic execution	klee.github.io	docs
Hoare logic intro · Software Foundations Vol 1	softwarefoundations.cis.upenn.edu	livro

H · Archive evolutivo (Darwin Godel + MAP-Elites)

Inovacao Sakana 2025. Reference implementation publica.

RECURSO	LINK	TIPO
Darwin Godel Machine paper · Sakana 2025	arXiv 2505.22954	paper
DGM reference implementation	jennyzzt/dgm	github
Sakana AI blog · Darwin Godel	sakana.ai/dgm	blog

MAP-Elites paper · Mouret & Clune 2015	arXiv 1504.04909	paper
POET (Open-Ended Learning) · Wang et al	arXiv 1901.01753	paper
STOP paper (self-improvement gone wrong)	arXiv 2310.02304	paper

I · Process Reward Models (verifier inline)

Inline verifier head. PRMs guiando tree search.

RECURSO	LINK	TIPO
"Let's Verify Step by Step" · OpenAI	arXiv 2305.20050	paper
GenPRM paper · AAAI 2026	arXiv 2504.00891	paper
AB-MCTS (adaptive branching) · NeurIPS 2025	arXiv 2503.04412	paper
ProcessBench	QwenLM/ProcessBench	github

J · Datasets cyber AI (referencias academicas)

Conhecer o landscape de benchmarks e dados.

RECURSO	LINK	TIPO
CyberGym paper · Berkeley ICLR 2026	arXiv 2506.02548	paper
CyberGym repo oficial	sunblaze-ucb/cybergym	github
Cyber-Zero paper	arXiv 2508.00910	paper
CTF-Dojo paper	arXiv 2508.18370	paper
ZeroDayBench paper	arXiv 2603.02297	paper
Awesome AI Pentest (curated list)	insidetrust/awesome-ai-pentest	github
PentestGPT (referencia)	GreyDGL/PentestGPT	github
HuggingFace datasets tag cybersecurity	huggingface.co/datasets	docs

K · vLLM + serving + speculative decoding

Inferencia em producao + speculative draft pro rollout RL.

RECURSO	LINK	TIPO
vLLM docs	docs.vllm.ai	docs
vLLM repo	vllm-project/vllm	github
PagedAttention paper	arXiv 2309.06180	paper
SGLang (alternativa)	sgl-project/sglang	github
Speculative decoding survey	arXiv 2401.07851	paper
llama.cpp · GGUF + edge inference	ggerganov/llama.cpp	github

L · World models + mental simulator (Tier 3 inovacao)

Continued pretrain em execucao traces. World models.

RECURSO	LINK	TIPO
World Models paper · Ha & Schmidhuber	arXiv 1803.10122	paper
DreamerV3 paper	arXiv 2301.04104	paper
Genie 2 · DeepMind 2024	deepmind.google blog	blog

M · State machine policies + agent harness

DSL pra forcar disciplina no Crafter module.

RECURSO	LINK	TIPO
DSPy · Stanford NLP	stanfordnlp/dspy	github
Guidance · Microsoft	guidance-ai/guidance	github
LangGraph · agent state machines	langchain-ai/langgraph	github
OpenAI Agents SDK 2026	openai/openai-agents-python	github
ReAct paper	arXiv 2210.03629	paper

6. Ferramentas e contas a abrir

Contas necessarias (5 fundadores em paralelo)

SERVICO	QUANTIDADE	CUSTO	PRA QUE
Kaggle	5 contas (1 por fundador), phone-verified	gratis	20h/sem T4 cada = 100h/sem total
Google Cloud · Colab Pro+	20 contas (4 por fundador)	\$50/mes × 20 = \$1000/mes	3h/dia × 20 contas = 420h/sem T4 + A100 esporadico
Modal	1 conta lab	~\$300/mes uso esperado	CPU host pra sandbox (gdb + gcov + counterfactual + SMT)
HuggingFace	1 conta org <code>iterate-labs</code>	gratis	Empurrar adapter cada 200 passos · publicar modelos
W&B	1 conta org <code>iterate-labs</code>	gratis academic	Tracking todas as runs
GitHub (ja temos)	5 contas · org <code>iterate-labs-ai</code>	gratis	Codigo + issues + Actions
Wasabi ou Backblaze B2	1 bucket	~\$50/mes (5TB)	Checkpoint storage S3-compatible
Google for Startups (alvo)	1 application	\$25k credits	Submit antes 27 jun, chega ago
AWS via NVIDIA Inception (alvo)	1 application	\$100k tier	Inception registra free, AWS routing fim ago

Software stack (instalar na primeira sessao)

```
pip install -U unsloth trl transformers accelerate peft bitsandbytes
pip install verl # quando comecar RL principal (sprint 3)
pip install vllm # inferencia
pip install datasets pyyaml wandb huggingface_hub
pip install scrapling[fetchers] # scraping
pip install z3-solver # SMT validator
pip install qdrant-client # primitive library vector index

# Sandbox host (Modal)
apt-get install -y gdb gdbserver gcc-multilib lcov afl++ \
    z3 cppcheck clang clang-tools
docker pull aflplusplus/aflplusplus:latest
docker pull gcr.io/oss-fuzz-base/base-runner
```

7. Sprints sem-a-sem

Sprint 1 · dias 1-7 · Foundation jun 22 - jun 28

5 contas Kaggle + 20 contas Colab + 1 Modal funcionando. Decontamination pipeline 3-camadas. Held-out 30 CVE curado. CyberGym subset 200 containerizado. Baseline zero-shot Qwen Coder 3B documentado. Continued pretrain Caracal Base 3B em andamento (1.5B tokens).

Sprint 2 · dias 8-14 · 5 modulos SFT estagio 2 jun 29 - jul 5

Caracal Base publicado HF Hub. Cada fundador pega tarefa do pool e treina o modulo correspondente. Pool roda sem dono, conforme disponibilidade. 5 LoRAs no HF Hub ao fim do sprint. Mental Simulator head treinada. Inline Verifier Head treinada. Reward function v1 testada.

Sprint 3 · dias 15-22 · Treino por reforco principal jul 6 - jul 13

verl + DAPO rodando. 3500 passos GRPO sobre 5 modulos coordenados. Archive DGM com 450 celulas MAP-Elites populadas. Primitive library cresce. Speculative draft Phi-3-mini ativo. Avaliacao intermediaria CyberGym 1507 dia 20.

Sprint 4 · dias 23-30 · Avaliacao + zero-day + ship jul 14 - jul 21

Avaliacao final CyberGym 1507 + CyBench + HumanEval+ regression. Caça zero-day em 30 OSS held-out com responsible disclosure. Caracal-1 v1 publicado HF Hub com model card. Demo Colab publico GGUF quantizado. Preprint arXiv submit.

8. 35 tarefas (com prioridade)

MUST obrigatorio pra ship v1. **STRETCH** adicionar se folga. **v1.5+** depois do ship.
kernel-D 2 revisores.

Sprint 1 · Foundation

C1 Setup Kaggle 5 contas + Colab Pro+ 20 contas + Modal CPU host MUST

5 contas Kaggle phone-verified, 20 Google + Colab Pro+, 1 Modal lab. Smoke test 10 vulneraveis paralelo. `infra/compute_inventory.md` documentando 520h/sem.

C2 Bake-off Qwen-Coder-3B vs R1-Distill-1.5B vs Phi-4-mini-3.8B MUST

50 tarefas CyberGym slice, zero-shot. ADR `infra/decisions/0001-base-model.md` . Kill-switch >3pp uplift troca base.

C3 Baseline zero-shot Qwen Coder 3B CyberGym + probe 100

MUST

Numeros pass@1 e pass@5 em `infra/baselines/qwen_coder_3b_zero_shot.md` .

C4 Caracal Base 3B continued pretrain 1.5B tokens cyber

MUST

NVD + patch diffs + OSS-Fuzz + Project Zero + Trail of Bits + AFL++ + PrimeVul + BigVul + DiverseVul + Secure-Instruct + Juliet + SARD + Devign + VulDeePecker + CrossVul + MegaVul + Reposvul + PatchDB + SVEN. Unsloth + LoRA r=32 alpha=64. 80h T4. Publicar `iterate-labs/caracal-base-3b` .

C5 Held-out 30 CVE 2026 OSS para zero-day hunt

MUST

kernel-D

30 CVE pos jan/2026, C/C++, <10K LOC. Hash check zero overlap. `eval/held_out_2026.yaml` .

C6 Pipeline decontamination 3 camadas

MUST

kernel-D

SHA256 + 8-gram Jaccard >0.3 + cosseno embedding >0.85. CVE-ID dedup. Falsos positivos <2%.

C7 CyberGym subset 200 + sandbox Modal Docker hardened

MUST

200 vulns curadas, gVisor + firejail + nsjail + seccomp, 60s timeout, 2GB ram, rede off. 5 attack tests passam.

C8 Reward verifier-gated + anti-supressao + coverage

MUST

kernel-D

Binario r=1 se crash CWE match + patch passa testes + anti-supressao 10 inputs adjacentes. Coverage gcov. Bootstrap shaping 1500 passos.

C9 Archive Darwin Godel + MAP-Elites 450 celulas

MUST

30 CWE × 5 primitivas × 3 langs. Parent sampling score / (1 + filhos com edicao). Snapshot cada 500 passos.

C10 Kaggle 5 contas script rotacao

MUST

5 contas phone-verified, T4 ativo, script rotacao API kaggle.json.

C11 Colab Pro+ 20 contas script rotacao + auto-resume

MUST

20 contas ativadas. Auto-resume HF Hub checkpoint testado.

C12 Caracal Base 3B model card + estrategia reuso

MUST

Model card HF Hub completo. ADR reuso pra Forja-MLE, Visao-Driving, Voz-Duplex.

Sprint 2 · Tier 3 inovacoes

C13 Causal trace pretraining (mental simulator head)**MUST**

100K traces AFL/OSS-Fuzz (gcov + ASan dumps). Head 50M sobre Caracal Base. Self-supervised next-state. ~20h T4.

C14 AST attention overlay head**STRETCH**

Head 12M usa AST adjacency como bias. Parser tree-sitter pra C/C++. End-to-end com Stage 2.

C15 State machine policy DSL + validator**MUST**

6 estados ANALYZE / HYPOTHEZIZE / INSTRUMENT / EXECUTE / REFINE / PATCH. Validator harness valida transicoes.

C16 Speculative draft model setup (Phi-3-mini)**STRETCH**

Distillar Phi-3-mini em outputs Caracal Base. Plug em vLLM. 3-5x speedup rollouts.

C17 Counterfactual mutator + reward causal**MUST**

Pra cada PoC gerar mutacao minimal que NAO triggaria. Reward por disparidade. Modal CPU.

C18 Compositional primitive library (writeable RAG)**MUST**

Banco Qdrant indexado por embedding. Modelo escreve quando descobre primitive nova. Ferramenta `retrieve_primitive(query)`.

C19 Inline verifier head PRM 30M**MUST**

Cabeca adicional sobre embedding final. Dual loss gera + julga. Tree search guided.

Sprint 2-3 · 5 modulos especialistas (pool aberto)**C20 Modulo Recon · embodied gdb live****MUST**

gdb/mi protocol parser. Action space 50 comandos. Sandbox com gdbserver. SFT em 5K gdb traces. RL adapted pra Recon.

C21 Modulo Hypothesizer · 3 bug hypotheses**MUST**

SFT em 8K CTF writeups + Project Zero. Output 3 hipoteses estruturadas por target com CWE class + primitive + dificuldade.

C22 Modulo Crafter · state machine + PoC**MUST**

SFT em 5K pares PoC+patch + Cyber-Zero traces. Output usa state machine C15. Counterfactual reward C17.

C23 Modulo Validator · SMT bounded Z3**MUST**

SFT em pares spec+PoC sinteticos. Z3 depth 5. Reward por consistencia spec ↔ PoC ↔ execucao.

C24 Modulo Patcher · unified diff + anti-supressao

MUST

SFT em patches CVE-fix. Output unified diff. Anti-supressao 10 inputs adjacentes. Preserva coverage.

C25 Protocolo de mensagens inter-modulo + arbitragem

MUST

Schema JSON estruturado. Recon → Hypothesizer → Crafter → Validator → Patcher. Logging W&B.

Sprint 3 · Reforco

C26 verl + DAPO config + warmup 1500 passos

MUST

lr=3e-6, KL=0.002 BF16 ref, eps_low=0.2 eps_high=0.28, batch=32, 16 rollouts/prompt. Bootstrap shaping ungated.

C27 Reforco principal 3500 passos + archive snapshots

MUST

Shaping condicionado $r_{\text{binary}} > 0$ capped 0.15. Snapshot cada 500 passos. Avaliacao intermediaria dia 20.

C28 Kill switch self-play dia 14

MUST

Se ganho RL <2pp vs SFT em probe set, desliga self-play e ship single-agent.

Sprint 4 · Ship

C29 Avaliacao final CyberGym 1507 pass@5 + ensemble

MUST

k=5, temp 0.8, top_p 0.95. 3-vote ensemble. HumanEval+/MBPP+ regression. SWE-Bench-Verified Lite 50 sanity.

C30 Caca zero-day em 30 OSS held-out 2026

MUST

Modal Docker pool. Responsible disclosure (private + 90 dias embargo). CVE request quando confirmado.

C31 Release HF Hub iterate-labs/caracal-1-v1

MUST

5 LoRAs + heads + tokenizer + model card + decontamination report + reproducibility scripts.

C32 Demo Colab publico GGUF quantizado

MUST

llama.cpp Q4_K_M. Clean-run T4 free <5min. Badge Open in Colab. Gif 30s.

C33 Preprint arXiv

STRETCH

"Caracal-1: A Multi-Module 3B Specialist for Vulnerability Discovery via Verifiable Reward RL with Embodied Debugging". 8 paginas + appendix.

v1.5+ (apos ship)

C34 SAE features bonus reward + steering + detection

v1.5+

SAE 50M sobre ativacoes Caracal Base layer 18. Feature labeling 50 features cyber. Paper hook top-tier.

C35 Caracal-XS 500M auto-distilled WebGPU edge

v1.5+

Distillation v1 → 500M. Q4 GGUF. transformers.js + WebGPU. Dev local browser. Telemetry opt-in.

9. Compute e credits

FONTE	CAPACIDADE	QUANDO	USO
Kaggle (5 contas T4)	100h/sem	Disponivel ja	Caracal Base continued pretrain + SFT estagio 1
Colab Pro+ (20 contas)	420h/sem T4 + A100 esporadico	Disponivel ja	SFT estagio 2 + RL warmup + ablacoes
Modal CPU host	continuo \$0.40/h 32-core	Disponivel ja	Sandbox + gdb + gcov + counterfactual + SMT
Google for Startups	\$25k credits A100	Chega ago (submit antes 27 jun)	RL principal 3500 passos Sprint 3
NVIDIA Inception + AWS Activate	\$100k tier AWS	Chega fim ago	Scaling v2 + Caracal Pro 7B mes 2+
Wasabi/Backblaze storage	5TB ~\$50/mes	Disponivel ja	Checkpoint streaming S3-compatible

Schedule semanal de placa de video

SEMANA	CARACAL BASE	SFT ESTAGIO 2	RL WARMUP	RL MAIN	AVALIACAO
1 (jun 22-28)	80h T4 Kaggle	-	-	-	20h T4 probe
2 (jun 29-jul 5)	-	40h T4 + 20h A100	50h A100	-	30h A100

3 (jul 6-13)	-	-	-	290h A100 GCP	40h A100
4 (jul 14-21)	-	-	-	polish	100h A100 full + 0-day hunt

10. Riscos e mitigacoes

RISCO	MITIGACAO
1. Colapso recompensa esparsa primeiros 1500 passos	Bootstrap shaping ungated + easy-first curriculum + coverage signal denso
2. Teto contexto 32K em logic bugs cross-file	LSP + ctags + RAG retrieval. v2 hybrid Mamba drop-in.
3. Patch-suppression hack (catch-and-ignore)	Anti-supressao re-roda sanitizer em 10 inputs adjacentes
4. Cyber-Zero distribution gap	Cyber-Zero so SFT, NUNCA rollout RL. 100% real Docker pra reward.
5. Contaminacao train + eval	Pipeline 3 camadas C6 + dedup CVE-ID + auditoria manual sample
6. Quantizacao FP8 corrompe KL	Referencia BF16 nao FP8, KL=0.002
7. Sandbox overrun custo Modal	Observation replay cache 35-55% + budget alert \$200/dia
8. Self-play colusao	Pools congelados + tripwire + dia 14 kill switch
9. Mode collapse archive	MAP-Elites 450 celulas + Darwin Godel parent sampling
10. gdb embodied Recon dataset ralo	Trajectories sinteticas com gdb scripted batch. Fallback static analysis.
11. SMT bounded Validator falha em casos complexos	2 modos: SMT pra simple, empirico counterfactual pra complex
12. 5 contas Kaggle preempted simultaneo	Auto-resume HF Hub. Fallback Colab Pro+ pool 20 contas.

11. Harness recursivo (6 loops self-improving)

Caracal-Ω-Recursive opera em 6 loops aninhados. Cada loop muta um nível da pilha. Boundary intocavel: **kernel-D** (evaluator, sandbox, reward registry, budget enforcer, classes config). So humanos editam kernel-D. Os 5 loops abaixo dele evoluem por si proprios contra o evaluator congelado. Padrao Darwin Godel Machine (Sakana 2025) mais SICA (17→53% SWE Bench Verified) mais salvaguardas STOP-anti-reward-hacking.

KERNEL-D · intocavel pelo loop · 2 humanos editam
evaluator · sandbox · reward registry · budget enforcer · classes config · capability tier

Loop A · Continued Pretrain
one-shot · 1.5B tokens cyber + 200M traces · sai Caracal Base 3B

Loop B · SFT estagio 2
one-shot · 35K exemplos curados por modulo · sai 5 LoRAs especialistas

Loop C · Reinforcement Learning (recursivo)
verl + DAPO · archive Darwin Godel snapshot cada 500 passos · parent sampling nao-greedy

Loop D · Mutacao de scaffold (recursivo, dentro do archive)
prompts · tool budget · retrieval depth · state machine transitions · temperatura schedule · valida em dev slice

Loop E · Mutacao de reward shaping coeficientes (recursivo, kernel-D limit)
pesos α β γ δ · coeficientes de shaping bootstrap · NAO muta funcao reward · NAO muta verifier oracle

Loop F · Promocao humana de mutacoes consistentes para kernel-D (semanal, 2 assinaturas)

11.1 Loops em detalhe

LOOP	O QUE MUTA	QUEM APROVA	FREQUENCIA	VALIDADOR
A · Pretrain	Pesos do Caracal Base	2 humanos PR	1 vez (Sprint 1)	Probe set + HumanEval+ regression
B · SFT	Pesos LoRA de cada modulo	1 humano PR	1 vez por modulo (Sprint 2)	Probe set + slice CyberGym 50
C · RL	Pesos LoRA (GRPO/DAPO updates)	auto (loop) com gates de eval cada 500 passos	continuo Sprint 3	Eval gate suite + probes anti-hack + dev slice 64 vulns

D • Scaffold	Prompts, tool budget, retrieval depth, state machine transitions, temperatura	auto (loop) com Pareto improvement em dev slice	continuo apos 1000 passos RL	Improvement em 100-prompt dev slice (NUNCA CyberGym 1507)
E • Reward shaping	Coeficientes α β γ δ ϵ do reward composto	auto (loop) com sanity check anti-hack	continuo apos 1500 passos RL	Coverage probe + length probe + canary leak check
F • Promocao kernel	Mutacoes do Loop D/E que reincidentem com ganho 3+ vezes	2 humanos + ADR registrado	weekly review	Manual auditoria + held-out 30 CVE 2026 sample (read-only)

11.2 Boundaries de seguranca (kernel-D intocavel)

COMPONENTE	LOOP PODE MUTAR?	POR QUE
Evaluator suite (CyberGym 1507, held-out 30 CVE)	NUNCA	Sistema nao deve poder editar seu proprio teste (STOP precedent)
Reward function (verifier oracle binario)	NUNCA	Gameabilidade infinita se loop edita propria recompensa
Reward shaping coeficientes (α β γ δ ϵ)	SIM (Loop E)	Coeficientes auxiliares so. Funcao oracle preservada.
Sandbox config (gVisor, seccomp, network off)	NUNCA	Loop nao pode reduzir proprio isolamento
Budget enforcer (tokens, wall, USD)	NUNCA	STOP: sistema tentou remover proprio budget. Bloqueado por design.
Classes A/B/C/D config	NUNCA	Loop nao pode escalar privilegio proprio
Prompts e scaffolds	SIM (Loop D)	Classe A: edita livre. Auditoria via archive.
Tools e retrieval logic	SIM (Loop D)	Classe B: edita com 1 review.
Pesos do modelo	SIM (Loops A/B/C)	Classe C: 2 reviewers + 3 seeds + suite completa.

11.3 Detector STOP-pattern

Probe automatica roda em cada gate de eval. Detecta tentativas do loop de:

- Editar arquivos do kernel-D (rejeitado por classes config + Action C4)
- Aumentar proprio budget (rejeitado por enforcer kernel)
- Reduzir proprio isolamento (sandbox config nunca acessivel)
- Suprimir crashes em vez de patchar (anti-supressao check Action C8)
- Coletar canary strings inseridas em corpus (probe anti-leak)

Trigger = trajetoria descartada + flag em `harness/incidents/YYYY-MM-DD.md` + review humano.

12. Estrategia Kaggle: 5 contas relay race

Constraint Kaggle: cada conta 20h/sem T4, sessao max 9h, sistema arquivos efemero. Solucao: **vezamento coordenado** via HuggingFace Hub como buffer compartilhado. Cada conta treina 9h consecutivas, push checkpoint, proxima conta resume. Total 80h Caracal Base = 9 sessoes de ~9h distribuidas em 2-3 dias.

12.1 Protocolo de vezamento

```
# A. Pegar slot livre no schedule (planilha shared)
# B. Pull checkpoint mais recente
huggingface-cli download iterate-labs/caracal-base-pretrain \
  --revision step-XXXX --local-dir ./ckpt

# C. Resume treino N steps (no notebook Kaggle)
python train_continued_pretrain.py \
  --resume-from ./ckpt \
  --steps-to-run 5500 \
  --output ./ckpt-out \
  --wandb-resume

# D. Push novo checkpoint
huggingface-cli upload iterate-labs/caracal-base-pretrain \
  ./ckpt-out . --revision step-YYYY \
  --commit-message "step YYYY by Pedro session 2 jun 23"

# E. Atualizar SCHEDULE.md com timestamp + revision
# F. Sinalizar proximo no Discord #caracal-base-relay
```

12.2 Cronograma de vezamento Sprint 1 (Caracal Base 80h T4)

Cada fundador faz 2 sessoes nessa semana (~18h dentro do limite 20h). 5 fundadores × 2 sessoes = 10 sessoes × 9h = 90h, com folga.

SESSAO	CONTA	INICIO	FIM	STEPS	CHECKPOINT SAIDA
1	fundador A	seg 23 jun 09h BRT	seg 23 jun 18h	0 → 5500	step-5500
2	fundador B	seg 23 jun 14h	seg 23 jun 23h	5500 → 11000	step-11000
3	fundador C	seg 23 jun 19h	ter 24 jun 04h	11000 → 16500	step-16500
4	fundador D	ter 24 jun 00h	ter 24 jun 09h	16500 → 22000	step-22000
5	fundador E	ter 24 jun 05h	ter 24 jun 14h	22000 → 27500	step-27500
6	fundador A (2a)	ter 24 jun 10h	ter 24 jun 19h	27500 → 33000	step-33000
7	fundador B (2a)	ter 24 jun 15h	qua 25 jun 00h	33000 → 38500	step-38500
8	fundador C (2a)	ter 24 jun 20h	qua 25 jun 05h	38500 → 44000	step-44000
9	fundador D (2a)	qua 25 jun 01h	qua 25 jun 10h	44000 → 50000 (fim)	step-50000 final

Sessoes 1+2, 2+3, 3+4... sao consecutivas com 4-5h sobreposicao. Overlap permite que conta atual prepare proxima (download checkpoint para conta seguinte enquanto continua treinando). **Loop sem gap de placa de video.** Treino total terminado quarta 25 jun manha. Sobra a semana para Sprint 1 finalizar outras tarefas.

12.3 Sprint 2 SFT estagio 2 · 5 modulos paralelo

Diferente de Sprint 1: cada modulo treina em paralelo. Cada fundador roda 1 modulo durante a semana usando suas 20h. 5 modulos × ~30h cada = 150h, distribuido em 5 contas paralelas em ~3 dias.

```
# Cada fundador roda na propria conta Kaggle, sem revezamento
python train_lora_module.py \
  --base-model iterate-labs/caracal-base-3b \
  --module {recon|hypothesizer|crafter|validator|patcher} \
  --dataset {gdb_traces|ctf_writeups|poc_patch|spec_synth|patches_cve} \
```

```
--r 32 --alpha 64 --lr 2e-4 \
--output iterate-labs/caracal-1-{module}-lora
```

12.4 Coordenacao operacional

MECANISMO	ONDE	PRA QUE
SCHEDULE.md no repo caracal-1	github.com/iterate-labs-ai/caracal-1/SCHEDULE.md	Slot booking + revisions + ultima atualizacao
HuggingFace Hub revisions	iterate-labs/caracal-base-pretrain com tags step-N	Buffer compartilhado entre contas
Discord/Slack #caracal-relay	canal lab	Sinalizacao tempo real "step-22000 pushed, sua vez"
W&B run continuo	iterate-labs/caracal-base-pretrain	Loss curve uniforme entre contas, tracking unico
Pre-flight script	scripts/preflight.sh	Valida HF auth + pulling checkpoint OK + GPU disponivel antes de iniciar
Auto-resume on SIGTERM	scripts/sigterm_handler.py	Se sessao Kaggle morre, salva checkpoint final + push HF Hub

12.5 Conflito e recuperacao

SITUACAO	O QUE FAZER
Duas contas push simultaneo	HF Hub aceita ambos como revisions diferentes. Quem termina 2o reverte e resume do primeiro (perdas: 1 hora).
Conta preempted antes de push	SIGTERM handler salva final. Se nao salvou, proximo resume do ultimo HF Hub valido (perdas: ate 9h).
Loss curve diverge entre contas	Cancela ultimas N sessoes, resume de checkpoint sane. ADR registra incidente.
HF Hub rate limit	Backoff exponencial + Wasabi/Backblaze como fallback (mesmo protocolo S3).
Kaggle conta ban (raro)	4 contas continuam. Conta nova phone-verified entra apos 24h.

13. Como usar (instrucoes por fundador)

13.1 Setup inicial (uma vez no inicio)

```
# 1. Clone o repo
git clone https://github.com/iterate-labs-ai/caracal-1
cd caracal-1

# 2. Install dependencias
pip install -r requirements.txt

# 3. Login HuggingFace (token de escrita)
# pega em https://huggingface.co/settings/tokens
huggingface-cli login

# 4. Login Weights and Biases
# pega em https://wandb.ai/authorize
wandb login

# 5. Login Kaggle (baixa kaggle.json em kaggle.com/settings)
mkdir -p ~/.kaggle
cp ~/Downloads/kaggle.json ~/.kaggle/
chmod 600 ~/.kaggle/kaggle.json

# 6. Login Modal (se vai mexer em sandbox)
modal token new

# 7. Verifica tudo OK
bash scripts/preflight.sh
```

13.2 Workflow diario (cada manha)

1. Abrir [caracal-1 issues](#) e ver pool de tarefas livres
2. Escolher uma task com label `sprint-1` e sem assignee · prioridade: **MUST** antes de **STRETCH**
3. Comentar `claim` e auto-assign
4. Olhar SCHEDULE.md para confirmar slot de placa de video disponivel
5. Bookar slot proximo no SCHEDULE.md (PR pequena)
6. Abrir notebook Kaggle ou Colab, autenticar HF + W&B
7. Rodar pre-flight: `bash scripts/preflight.sh`
8. Iniciar tarefa

13.3 Workflow durante treino (cada sessao Kaggle/Colab)

1. Pull checkpoint atual: `huggingface-cli download iterate-labs/MODELO --revision step-XXXX`
2. Iniciar W&B run resumindo o run anterior: `wandb.init(resume='allow', id='caracal-base-pretrain')`
3. Rodar script de treino com `--resume-from ./ckpt`
4. SIGTERM handler ativo: se Kaggle/Colab desconecta, salva final automatico
5. Push checkpoint a cada 200 passos como precaution: `huggingface-cli upload`
6. Ao terminar 9h ou steps planejados: push final, atualiza SCHEDULE.md, sinaliza proximo no Discord

13.4 Workflow PR (cada code change)

1. Branch local: `git checkout -b cN-meu-fix`
2. Commitar mudancas com mensagem clara
3. Push: `git push -u origin cN-meu-fix`
4. Abrir PR linkando a issue: `Closes #N`
5. Pedir review de outro fundador que NAO escreveu o codigo
6. Issues tag `kernel-D` : 2 reviewers obrigatorios
7. Apos merge: comentar `done` na issue, fechar, claim proxima

13.5 Como rodar sandbox local (debug)

```
# Modal local development
modal serve sandbox/caracal_sandbox.py

# Testar 1 vulneravel
modal run sandbox/caracal_sandbox.py::run_poc \
    --vuln-id cybergym-poc-42 \
    --poc-payload "... "

# Testar 1 patch
modal run sandbox/caracal_sandbox.py::run_patch \
    --vuln-id cybergym-poc-42 \
    --patch-diff "... "

# Testar 5 attack tests (deve TODOS bloquear)
modal run sandbox/test_attacks.py
```

13.6 Como rodar avaliacao (probe ou full)

```
# Probe set (100 prompts internos, rapido)
python eval/run_probe.py \
    --adapter iterate-labs/caracal-1-crafter-lora \
```

```
--base iterate-labs/caracal-base-3b \
--out reports/probe-step-XXXX.json

# CyberGym slice 50 (medio)
python eval/run_cybergym.py --subset slice-50 --adapter ...

# Full CyberGym 1507 (so dia 20 e dia 28 do sprint)
python eval/run_cybergym.py --subset full-1507 --adapter ... \
--temp 0.8 --top-p 0.95 --k 5
```

13.7 Documentos importantes

DOC	O QUE TEM
README.md	Visao geral repo + setup
SCHEDULE.md	Slot booking de placa de video, ultima revision do modelo
HOWTO.md	Esta secao 13 do plano + extras
infra/decisions/	ADRs · 1 por decisao importante
infra/baselines/	Numeros baseline pra ancora
infra/compute_inventory.md	Inventario de placa de video + Modal CPU
eval/held_out_2026.yaml	30 CVE held-out kernel-D · NUNCA tocar pra treino
harness/decisions/	ADRs de mutacoes Loop F (kernel promotions)
harness/incidents/	STOP-pattern flags e auditorias

13.8 Comandos rapidos cheat sheet

```
# Status do treino
huggingface-cli list-files iterate-labs/caracal-base-pretrain | tail -5
wandb runs --project iterate-labs/caracal-base-pretrain | head

# Quem ta no slot agora
cat SCHEDULE.md | grep "in-progress"

# Loss curve atual
python scripts/plot_loss.py --run-id caracal-base-pretrain --last 24h

# Audit STOP-pattern
ls harness/incidents/ | tail -5

# Custo Modal acumulado
modal billing status
```

```
# Push HF Hub revision tagged
huggingface-cli upload iterate-labs/MODEL ./ckpt-out . \
  --revision step-XXXX --commit-message "..."
```

13.9 Quem chamar quando

SITUACAO	QUEM
Treino travou em GPU/Kaggle/Colab	Discord #caracal-relay + screenshot do erro
HF Hub push falhou	Discord #caracal-relay + log
Slot conflict	Quem booking primeiro wins. Quem perdeu re-book proximo livre.
Bug no codigo critico (kernel-D)	Abre issue urgente · 2 fundadores debug juntos
STOP-pattern flag	Trigger automatico: para todo treino. 2 humanos review antes de retomar.
Sandbox attack test falhou	BLOCKER. Para tudo. 2 reviewers + ADR.
Modal custo >\$200/dia	Auto alerta. Pausa sandbox. Review configuracao.

14. Mensagem pronta pra enviar aos fundadores

Texto pronto para copiar e enviar no Discord ou WhatsApp do time. Direto ao ponto. Setup em 20 minutos. Bora trabalhar.

Bora rodar Caracal-1, pessoal.

Repo: <https://github.com/iterate-labs-ai/caracal-1>
Branch default: dev

Primeira coisa: aceitar convite que ja mandei pra org iterate-labs-ai no GitHub. Quem ainda nao aceitou, ve no email ou em <https://github.com/orgs/iterate-labs-ai/invitations>

Depois disso, fazer setup em 20 minutos:
<https://github.com/iterate-labs-ai/caracal-1/blob/dev/SETUP.md>

Passos rapidos:

1. `git clone https://github.com/iterate-labs-ai/caracal-1`
`cd caracal-1`
`git checkout dev`
2. `pip install -r requirements.txt`
3. Criar conta Hugging Face (huggingface.co/join) e me avisar pra mandar convite org iterate-labs
4. Criar conta Weights and Biases (wandb.ai/signup), idem
5. Phone-verify conta Kaggle (1 conta sua basta) e ativar GPU T4
6. Criar 4 contas Google + assinar Colab Pro+ em cada (R\$1000/mes total time, divide)
7. `bash scripts/preflight.sh` para confirmar tudo OK

Quando passar pelo preflight, ler nessa ordem:

- README.md (2 min)
- HOWTO.md (5 min, workflow diario)
- SCHEDULE.md (2 min, slot booking placa de video)
- CONTRIBUTING.md (5 min, PR + KERNEL-D rules)

Pra pegar tarefa:

<https://github.com/iterate-labs-ai/caracal-1/issues>

- Filtrar por label sprint-1 + sem assignee + MUST
- Comentar "claim" e pegar

Pra rodar treino na sua sessao de Kaggle:

- Bookar slot livre em SCHEDULE.md via PR pequena
- Abrir `train/notebooks/kaggle_continued_pretrain.ipynb` no Kaggle
- Editar 5 linhas iniciais (`SESSION_NUMBER`, `RESUME_REVISION`, `OUTPUT_REVISION` etc)
- Configurar Kaggle Secrets `HF_TOKEN` e `WANDB_API_KEY`
- Run all
- ~9 horas depois push final automatico no HF Hub
- Atualizar SCHEDULE.md trocando pending pra done

Pra Colab Pro+ mesma coisa em `train/notebooks/colab_continued_pretrain.ipynb`

Modelo: Caracal-1, especialista cyber 3B parametros. Base Qwen2.5-Coder-3B + 1.5B tokens continued pretrain proprio. 5 modulos LoRA (Recon, Hypothesizer, Crafter, Validator, Patcher). Ship v1 em 21 jul.

Pool aberto. Sem dono. Todos treinam tudo. Quem termina uma tarefa pega proxima do topo da fila.

Alvo: 40 a 50 por cento pass@5 em CyberGym 1507, 10 a 15 zero-days em 30 OSS held-

out, 700 dolares por zero-day amortizado.

Plano completo: <https://github.com/iterate-labs-ai/caracal-1> + docs/

Duvida no setup: chama no Discord canal #caracal-help (criar canal quando todos entrarem).

Quando todos tiverem rodado preflight com sucesso, fazemos kickoff call segunda 09h BRT para alinhar primeira sessao de revezamento.

Bora.

14.1 Versao curta (WhatsApp - ate 500 chars)

Caracal-1 bora.

Repo: github.com/iterate-labs-ai/caracal-1 (branch dev)

Setup 20min: github.com/iterate-labs-ai/caracal-1/blob/dev/SETUP.md

1. Aceita convite [iterate-labs-ai](#) no GitHub
2. Clone + `pip install -r requirements.txt`
3. Criar HF + WandB + Kaggle phone-verified + 4 Colab Pro+
4. `bash scripts/preflight.sh`

Quando passar, le README + HOWTO + SCHEDULE + CONTRIBUTING.

Tarefas em /issues, claim a sua. Treino em notebooks template.

Ship Caracal-1 v1 em 21 jul. Alvo 40-50% pass@5 CyberGym, 10-15 zero-days.

Kickoff seg 09h BRT.

14.2 Checklist primeiro dia para cada fundador

Imprimir e marcar conforme avanca. Tempo total: ~30 min.

#	ACAO	TEMPO	OK
1	Aceitar convite GitHub org iterate-labs-ai	1 min	☐
2	Criar conta HuggingFace (huggingface.co/join)	2 min	☐
3	Enviar handle HF pro Pedro pra incluir na org iterate-labs	1 min	☐

4	Criar conta W&B (wandb.ai/signup)	2 min	☐
5	Enviar handle W&B pro Pedro	1 min	☐
6	Conta Kaggle phone-verified · ativar T4	3 min	☐
7	Criar 4 contas Google (alias) · assinar Colab Pro+ em cada · \$200/mes seu	10 min	☐
8	<code>git clone</code> + <code>pip install -r requirements.txt</code>	3 min	☐
9	Login local HF, W&B, Kaggle	2 min	☐
10	Rodar <code>bash scripts/preflight.sh</code> · tudo verde	1 min	☐
11	Ler README + HOWTO + SCHEDULE + CONTRIBUTING	10 min	☐
12	Claim primeira issue em /issues label <code>sprint-1</code> + <code>MUST</code>	2 min	☐

14.3 Links diretos pra cada fundador acessar

LINK	O QUE TEM
github.com/iterate-labs-ai/caracal-1	repo principal · branch dev default
SETUP.md	passo a passo 20 min onboarding
HOWTO.md	workflow diario apos setup
SCHEDULE.md	slot booking placa de video relay race
CONTRIBUTING.md	PR style + KERNEL-D rules
docs/architecture.md	arquitetura modelo 5 min
docs/recursive_harness.md	6 loops self-improving
docs/learning_trail.md	links de aprendizado (consulta)
/issues	pool de tarefas, claim sua
Kaggle notebook	plug-and-play template treino
Colab notebook	plug-and-play template treino
huggingface.co/iterate-labs	org HF · pesos publicados aqui
wandb.ai/iterate-labs	org W&B · tracking treino

15. Glossario

TERMO	SIGNIFICADO
LoRA	Low-rank adapter. Treina apenas matrizes pequenas adicionais congelando base. $r=32$ = posto 32. ~1% dos parametros do total.
QLoRA	LoRA quantizado. Base em 4-bit + adapter 16-bit. Cabe 7B em 16GB.
SFT	Supervised Fine-Tuning. Treino supervisionado pares (input, output).
Continued pretrain	Mid-training. Continua treino com corpus novo antes do SFT.
GRPO	Group Relative Policy Optimization. Algoritmo RL do DeepSeek-R1.
DAPO	Variante GRPO 2026 com clip assimetrico $eps_low=0.2$ $eps_high=0.28$.
Reward verificavel	Recompensa binaria executavel. Imune a reward hacking.
PoC	Proof of Concept. Codigo curto que reproduz vulnerabilidade.
CVE	Common Vulnerabilities and Exposures. Catalogo publico de vulns.
CWE	Common Weakness Enumeration. Classificacao de tipos de bug.
ASan/UBSan/MSan	Sanitizers do gcc/clang que detectam OOB, undefined behavior, mem nao init.
gdb/mi	Machine Interface do gdb. Output estruturado parseable.
gcov	Coverage tool do gcc.
OSS-Fuzz	Servico Google fuzzing continuo em OSS critico.
Darwin Godel Machine	Arquivo evolutivo de checkpoints. Sakana 2025.
MAP-Elites	Quality-diversity. Guarda melhor por combinacao de comportamentos.
vLLM	Motor inferencia com PagedAttention. Default da industria.
Counterfactual	Mutacao minimal que NAO triggaria bug. Sinal causal.
SMT	Satisfiability Modulo Theories. Solver logica primeira ordem (Z3).
SAE	Sparse Autoencoder. Decompoe ativacoes em features interpretaveis.

CyberGym

Benchmark Berkeley ICLR 2026. 1507 vulns / 188 projetos.

HuggingFace Hub

Plataforma pra publicar modelos. Equivalente GitHub pra pesos.

Doc unico Caracal v1 plano executavel. Pool aberto, todos treinam tudo. Repos: github.com/iterate-labs-ai. Tasks live: caracal-1/issues.